MODELOS DE LINGUAGEM E GENERATIVOS

A Arquitetura Transformer & O Mecanismo de Atenção

O Gargalo Histórico das RNNs e o Artigo 'Attention Is All You Need'

Limitações das RNNs e LSTMs:

- **Processamento Estritamente Sequencial:**
  O token t só pode ser processado após o token t-1. Impossibilita o paralelismo em GPUs.
- **Desvanecimento do Gradiente (Vanishing Gradient):**
  Informações do início da frase são 'esquecidas' à medida que a sequência cresce.
- **Gargalo do Vetor de Contexto Fixo:**
  Forçar todo o significado de uma frase longa em um único vetor oculto de tamanho fixo.

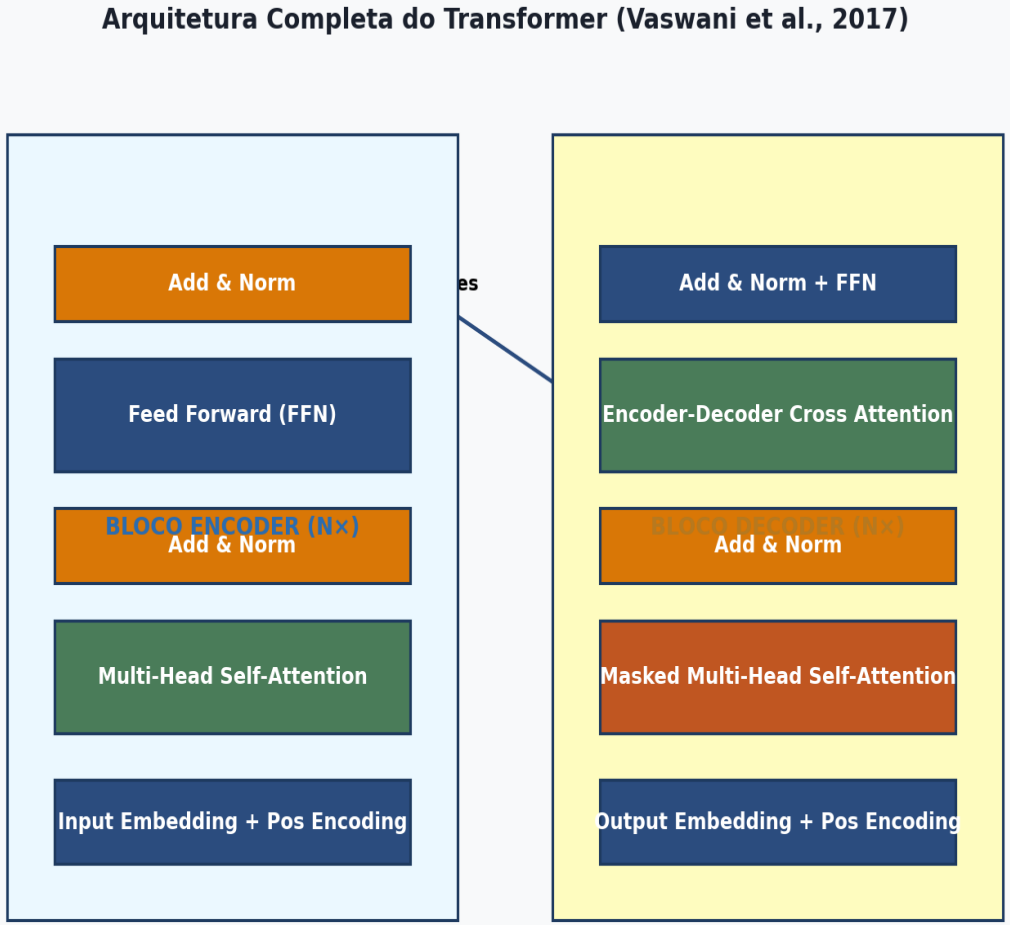
A Solução Transformer (Vaswani et al., 2017):

- **Eliminação Total da Recorrência:**
  Toda a sequência de tokens é alimentada de uma só vez na rede neural.
- **Paralelismo Massivo em GPU:**
  Treinamento até 100x mais rápido em volumes gigantescos de dados da web.
- **Conexões Diretas entre Qualquer Par de Tokens:**
  O mecanismo de Atenção calcula a relação de qualquer palavra com qualquer palavra em tempo O(1).

Visão Geral da Arquitetura Encoder-Decoder

A Estrutura de Dois Blocos:

- **Encoder (Codificador - Esquerda):**
  Recebe o texto de entrada, aplica Self-Attention bidirecional e gera representações contextuais ricas.
- **Decoder (Decodificador - Direita):**
  Gera a saída autorregressivamente. Usa Masked Self-Attention (para não olhar o futuro) e Cross-Attention (para consultar o Encoder).



Entrada & Incorporação Posicional (Positional Encoding)

O Desafio da Ordem no Transformer:

- **Ausência de Sequência Nativa:**

  Como o Transformer processa todos os tokens simultaneamente, ele trataria 'O gato comeu o peixe' identicamente a 'O peixe comeu o gato'.

- **Solução por Injeção de Posição:**

  Somar ao vetor da palavra (Token Embedding) um vetor de posição (Positional Embedding) com a mesma dimensão d_model.

- **Equação do Vetor Final de Entrada:**

  Vetor Entrada_i = TokenEmbedding(palavra_i) + PositionalEncoding(pos_i)

Funções Seno e Cosseno (Vaswani 2017):

- **Fórmulas Trigonométricas:**

  PE(pos, 2i) = sin(pos / 10000^(2i / d_model))

  PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))

- **Vantagens das Funções Harmônicas:**

  1. Valores sempre limitados entre -1.0 e +1.0 (não estouram a escala do vetor de palavras).

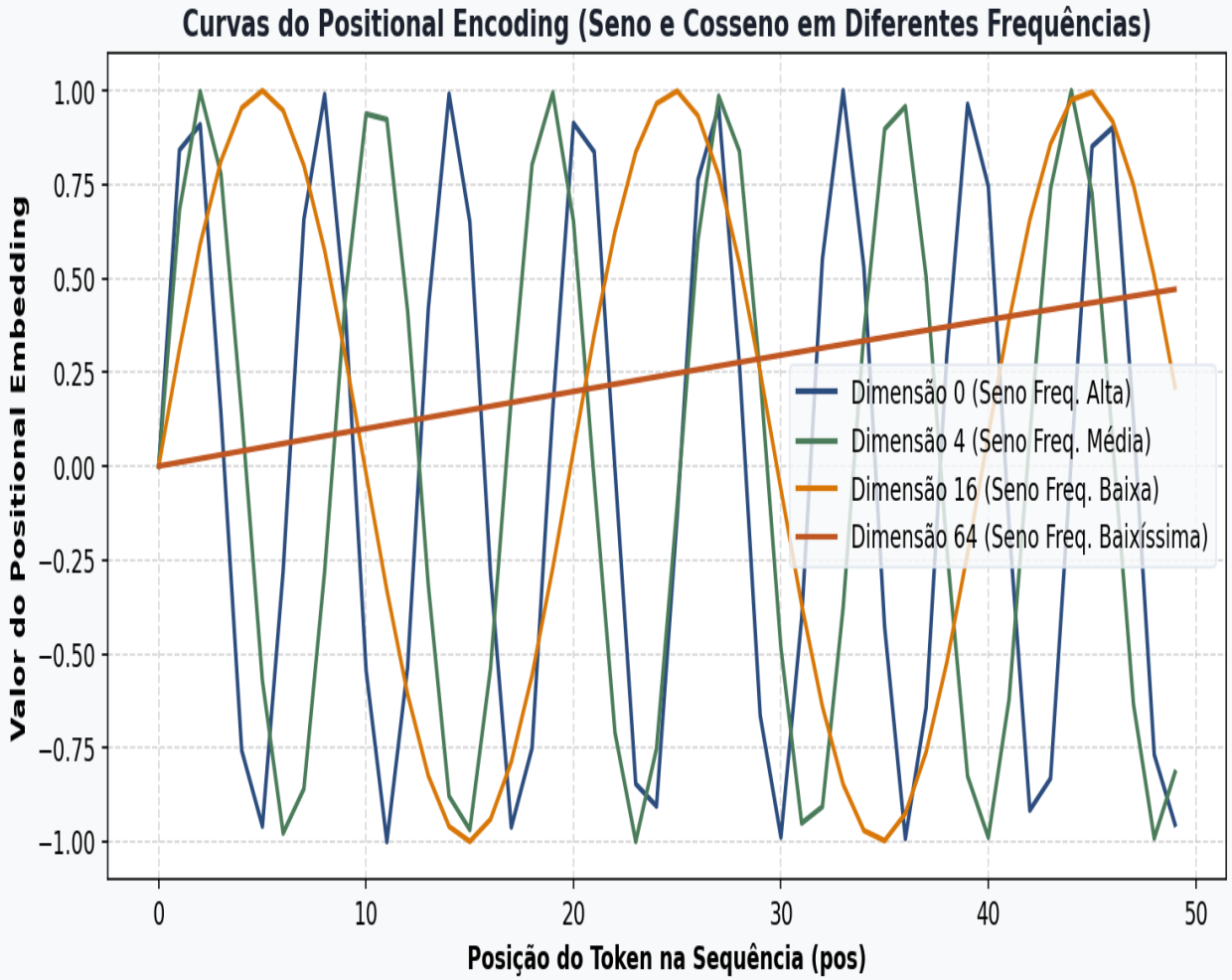
  2. Permitem ao modelo inferir posições relativas (|i - j|) através de rotações lineares.

  3. Generalizam para comprimentos de sequência não vistos no treino.

Visualização Curvas de Frequência do Positional Encoding

Entendendo o Mapeamento Harmônico:

- **Analogia do Relógio/Hodômetro:**
  As dimensões iniciais (ex: Dim 0) variam rapidamente (ponteiro dos segundos). As dimensões finais variam devagar (ponteiro das horas).
- **Assinatura Posição Única:**
  A combinação de valores em todas as dimensões cria uma 'coordenada espacial' exclusiva para cada token.
- **Deslocamento sem Destruição:**
  Em um espaço de 768 dimensões, somar o vetor posicional altera a 'direção' para indicar a ordem, sem apagar o significado da palavra.



O Mecanismo de Atenção: O que é Self-Attention?

Conceito de Auto-Atenção (Self-Attention):

- **Definição Fundamental:**

  Mecanismo que permite a cada token em uma frase 'olhar para' e ponderar a importância de todos os outros tokens da mesma frase.

- **Superando Vetores Estáticos (Word2Vec):**

  No Word2Vec, a palavra 'manga' tem sempre o mesmo vetor. Na Self-Attention, o vetor de 'manga' muda dinamicamente se o contexto contiver 'camisa' ou 'fruta'.

- **Pontuação Dinâmica de Relevância:**

  A rede aprende matrizes de pesos que determinam quanto a palavra A deve 'absorver' da palavra B.

Exemplo Prático (Alammar & Grootendorst):

- **Resolução de Ambiguidade Pronominal:**

  Frase: 'O animal não atravessou a rua porque ELE estava muito cansado.'

- **O Papel do Mecanismo de Atenção:**

  Ao processar o token 'ELE', o mecanismo de Self-Attention calcula pontuações para todos os tokens da frase.

- **Resultado da Atenção:**

  A palavra 'ELE' atribui a maior porcentagem de atenção ao token 'animal' (e não à 'rua'). O vetor de 'ELE' é atualizado com o contexto do animal!

A Tríade Fundamental: Query (Q), Key (K) e Value (V)

Analogia de Busca em Banco de Dados / Arquivo:

- Query (Q - Consulta): A 'pergunta' feita pelo token atual ('O que eu preciso buscar no contexto?').
- Key (K - Chave): O 'rótulo/etiqueta' que cada token oferece ('Qual informação eu possuo?').
- Value (V - Valor): O 'conteúdo real' do token que será transmitido se a Chave for compatível com a Query.

Como os Vetores Q, K, V são Criados no Modelo:

1. Matrizes de Projeção Treináveis (W_Q, W_K, W_V):

Cada vetor de entrada X (dimensão d_model) é multiplicado por 3 matrizes de pesos aprendidas no treinamento:

$$Q = X \cdot W_Q \quad | \quad K = X \cdot W_K \quad | \quad V = X \cdot W_V$$

2. Separação de Papéis Funcionais:

Permite que o mesmo token desempenhe papéis completamente diferentes ao 'perguntar' (Q), ao 'se oferecer' (K) e ao 'entregar conteúdo' (V).

Fluxo da Atenção Produto Escalar Escalada (Scaled Dot-Product)

As 4 Etapas do Cálculo de Atenção:

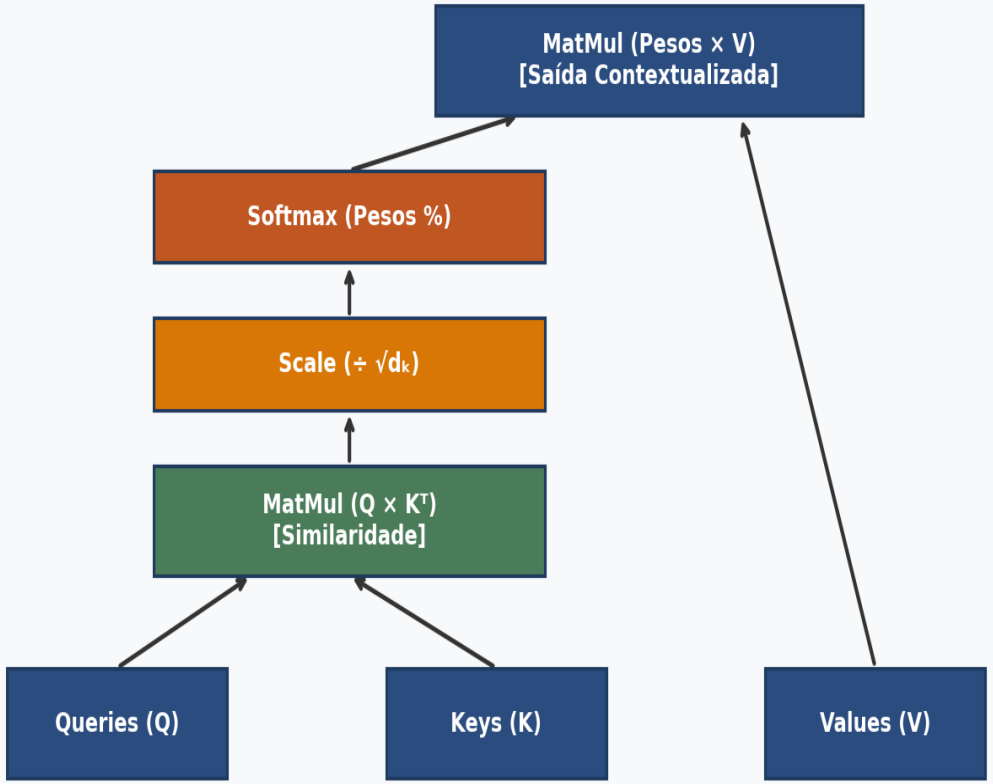
1. MatMul ($Q \times K^T$):
Produto escalar mede a similaridade/afinidade entre todas as Queries e todas as Keys.

2. Escalonamento ($\div \sqrt{d_k}$):
Evita que o produto escalar atinja valores gigantes em dimensões altas, impedindo a saturação do Softmax com gradientes nulos.

3. Softmax:
Converte pontuações brutas em probabilidades/porcentagens de atenção (soma 1.0 por linha).

4. MatMul (Pesos × V):
Pondera os Valores V para criar o novo vetor contextualizado.

Mecanismo de Atenção Produto Escalar Escalada (Scaled Dot-Product Attention)



A Fórmula Fundamental do Transformer & Exemplo Prático

A Fórmula Mestra (Vaswani et al., 2017):

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Exemplo Prático: 'O algoritmo opera em três etapas'

- **Processando o token 'opera' na posição 3:**

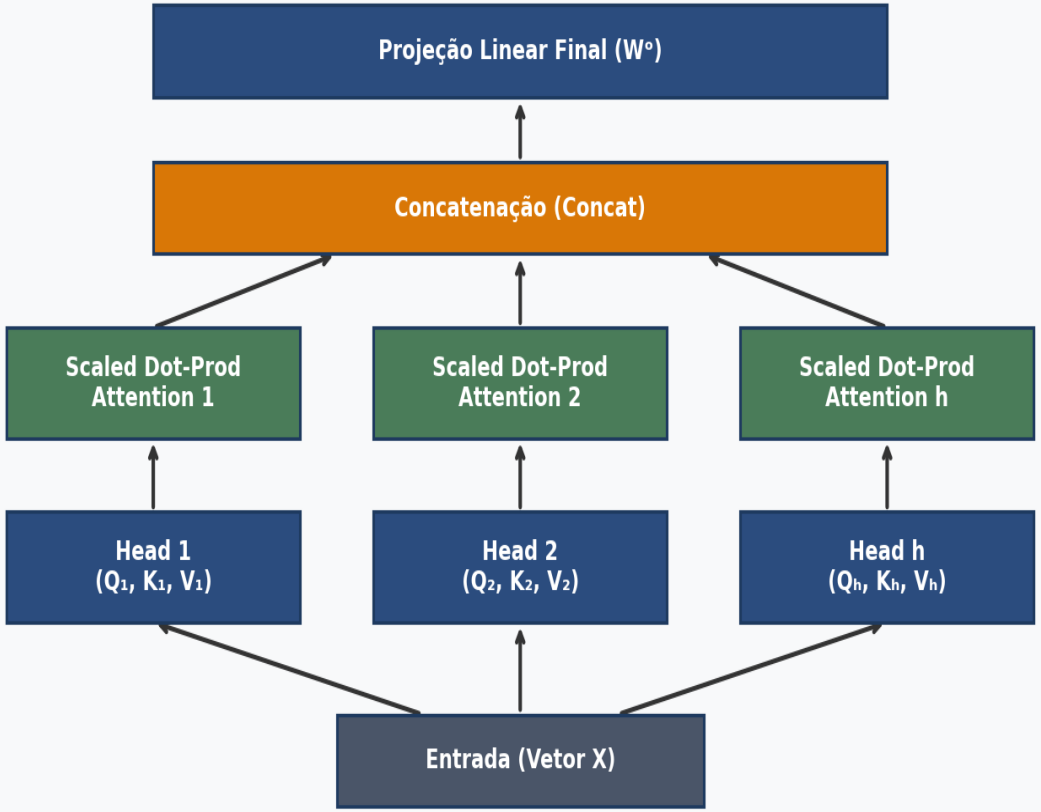
1. A Query de 'opera' faz a pergunta: 'Quem é o meu sujeito que realiza a ação?'
2. O produto escalar Q_opera · K_all calcula a compatibilidade com todos os tokens:
   - Nota com 'O': 0.1 | Nota com 'algoritmo': 8.5 | Nota com 'opera': 1.2
3. O Softmax converte as notas em porcentagens: 'O': 2% | 'algoritmo': 88% | 'opera': 10%.
4. **Multiplicação por V: O novo vetor de 'opera' recebe 88% da informação de 'algoritmo'. Agora o modelo sabe exatamente que é um algoritmo executando uma ação computacional!**

Atenção Multi-Cabeça (Multi-Head Attention)

Por que Usar Múltiplas Cabeças?

- **A Limitação de 1 Única Cabeça:**
  Uma única atenção tende a focar em apenas uma relação dominante por vez (ex: substantivo-verbo).
- **Múltiplos Subespaços de Projeção:**
  Dividir d_model em h cabeças paralelas (ex: h=8 no GPT-2, h=128 no GPT-4). Cada cabeça projeta Q, K, V com suas próprias matrizes W.
- **Especialização de Relações:**
  - Head 1: Foca em concordância gramatical.
  - Head 2: Foca em referências pronominais.
  - Head 3: Foca em entidades/marcas temporais.

Arquitetura de Atenção Multi-Cabeça (Multi-Head Attention)



Concatenação e Projeção Linear de Múltiplas Cabeças

A Mecânica de Fusão de Informação das Cabeças:

1. Execução Paralela:

Cada uma das h cabeças calcula independentemente: head_i = Attention(Q · W_i^Q, K · W_i^K, V · W_i^V)

2. Concatenação Lado a Lado (Concat):

As matrizes de saída de todas as h cabeças são unidas horizontalmente em um único vetor grande.

3. Projeção Linear Final (W^O):

Multiplica-se a matriz concatenada pela matriz treinável W^O para projetar o resultado de volta à dimensão d_model original.

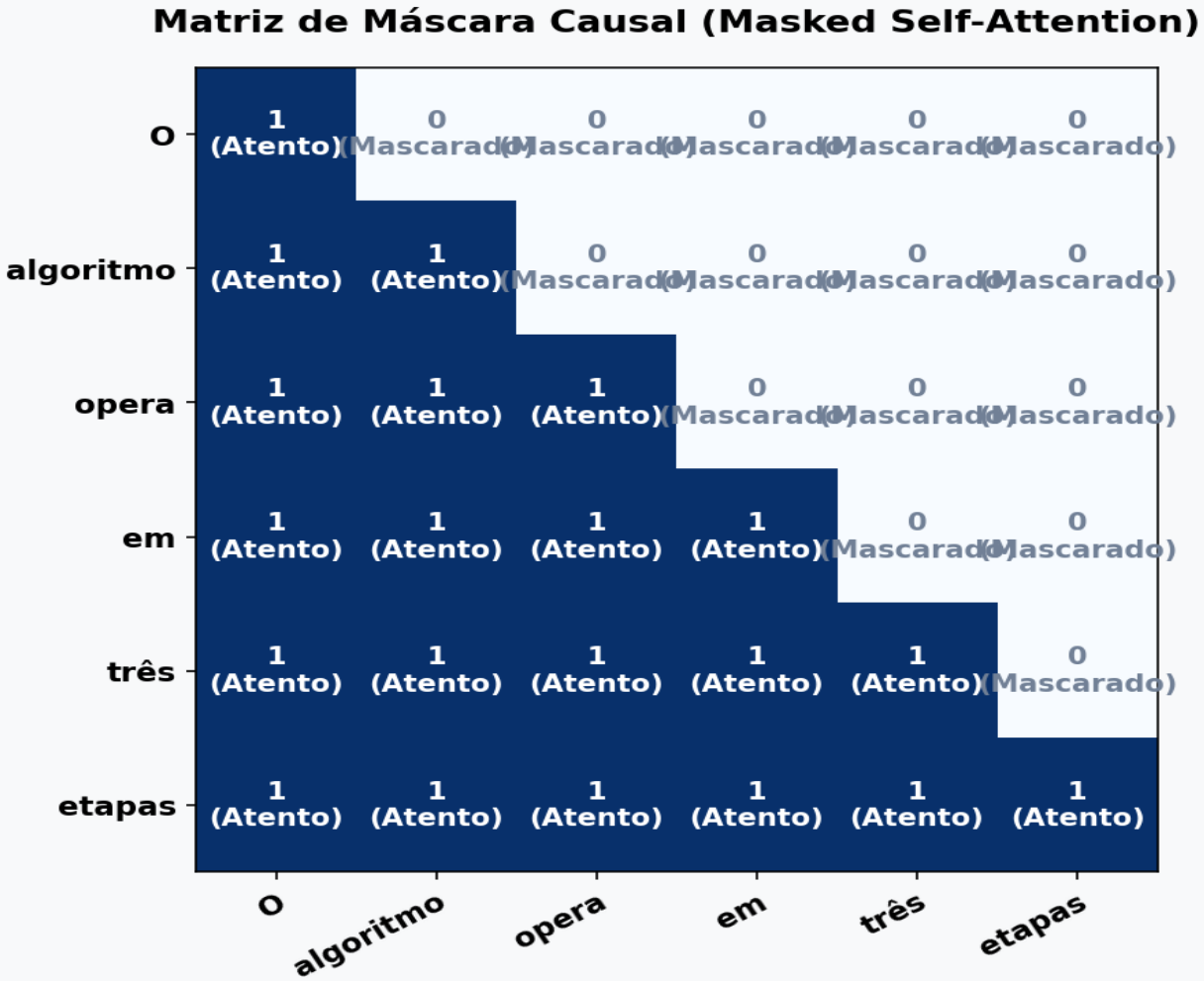
- **Formulação Completa:**

MultiHead(Q, K, V) = Concat(head_1, head_2, ... , head_h) · W^O

Atenção com Máscara Causal (Masked Self-Attention)

O Desafio Autorregressivo no Decoder:

- **Vazamento de Informação no Treino:**
  No treinamento, a GPU recebe toda a frase de saída. Sem restrição, a palavra no tempo t 'colaria' e leria o token do tempo t+1.
- **Aplicação da Máscara Causal:**
  Antes do Softmax, adiciona-se -∞ na matriz de pontuações $Q \cdot K^T$ para todas as posições à direita.
- **Efeito Matemático do -∞:**
  Como e^-∞ = 0, a probabilidade de atenção para tokens futuros torna-se EXATAMENTE 0%. Garantia de causalidade perfeita!



As Três Famílias Derivadas do Transformer

1. Encoder-Only

- **Exemplos: BERT, RoBERTa, DeBERTa.**
- Tipo de Atenção: Bidirecional irrestrita.
- Treinamento: Masked Language Model (MLM).
- Aplicações: Classificação de texto, busca semântica, extração de entidades (NER).

2. Decoder-Only

- **Exemplos: GPT-3/4, Llama 2/3, Gemini, Mistral.**
- Tipo de Atenção: Mascarada Causal (autorregressiva).
- Treinamento: Causal Language Modeling (prever próximo token).
- Aplicações: Chatbots, geração de artigos, criação de código e escrita criativa.

3. Encoder-Decoder

- **Exemplos: T5, Flan-T5, BART.**
- Tipo de Atenção: Bidirecional no Encoder + Causal & Cross no Decoder.
- Treinamento: Seq2Seq (Span Corruption).
- Aplicações: Tradução automática, sumarização de documentos.

Da Saída do Decoder ao Próximo Token: Projeção & Amostragem

O Processo de Decodificação do Próximo Token:

1. Saída Bruta do Decoder:

O último bloco do Decoder entrega um vetor denso contextualizado de dimensão d_model.

2. Camada Linear Final (Unembedding):

Projeta o vetor d_model para uma dimensão igual ao tamanho do vocabulário (ex: 50.257 palavras no GPT-2, 32.000 no Llama). Produz os 'Logits' brutos.

3. Normalização Softmax Final:

Converte os Logits em uma distribuição de probabilidade válida sobre todas as palavras do dicionário.

4. Estratégias de Seleção / Amostragem (Sampling):

- Greedy Search: Escolhe sempre o token de maior probabilidade (pode ser repetitivo).
- Temperatura T: Ajusta a suavidade da distribuição (T próximo de 0 = determinístico; T alto = criativo).
- Top-K & Top-P (Nucleus): Restringe a escolha aos K tokens mais prováveis ou à massa cumulativa P.

Quadro Comparativo Global das Arquiteturas de Linguagem

Critério	RNN / LSTM	Word2Vec	Transformer Encoder	Transformer Decoder
Processamento	Sequencial (O(N)) Lento em GPU	Matriz Estática Sem contexto	Paralelo (O(1)) Ultra-rápido GPU	Paralelo no Treino Autorregressivo na Inferência
Contexto	Limitado / Vanishing Unidirecional	Nenhum (Vetor Fixo estático)	Bidirecional Global Completo (Irrestrito)	Mascarado Causal (Apenas tokens passados)
Mecanismo	Recorrência Oculta (Hidden State)	Skip-Gram / CBOW Lookup Table	Multi-Head Self-Attention (Q, K, V)	Masked Self-Attention + Cross-Attention
Aplicações	Séries Temporais PLN Clássico antigo	Classificação simples Search estático	BERT, Classificação, Busca Semântica, NER	GPT, Llama, Gemini, Chatbots, Geração de Código

Referências Bibliográficas & Leituras Recomendadas

Leituras recomendadas:

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NIPS 2017).

2. Alammar, Jay, & Grootendorst, Maarten. (2024). Hands-On Large Language Models: Language Understanding and Generation. O'Reilly Media. (Capítulo 3: Transformer Architectures).

3. DataCamp Tutorial. (2024). How Transformers Work: A Detailed Exploration of Transformer Architecture.

4. Wolfram, Stephen. (2023). What Is ChatGPT Doing ... and Why Does It Work? Writings of Stephen Wolfram.

5. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv:1810.04805.